601.433/633 Introduction to Algorithms **Fall 2026**
Homework #5 **Due:** October 22, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Completing Homeworks (50 points)

Suppose (hypothetically) that you were taking a class, possibly called "Introduction to Algorithms", in which the homeworks were extremely difficult. After enough complaining, the professor decided to make the following changes. There are two homework assignments each week rather than one, an "easy" assignment and a "hard" assignment. The hard assignment is worth more points, but it is in fact so difficult that you can only complete it if you're completely rested and prepared, meaning that you cannot have done either of the assignments the week before.

More formally, let n be the number of weeks in the class, let h_i be the number of points for the hard assignment in week i, and let e_i be the number of points for the easy assignment in week i. Note that h_i does not have to be equal to h_j for $i \neq j$ (although it might be), and similarly with e_i and e_j. Assume that you know all of these values in advance. Then the goal is compute a schedule which in each week tells you whether to do nothing, the easy assignment, or the hard assignment and maximizes the total number of points, subject to the restriction that if you do a hard assignment in week i you cannot have done *any* assignment in week $i-1$.

(a) (15 points) One obvious approach would be to choose a hard assignment in week i if we get more points than if we completed the easy assignments for weeks i and $i-1$. This would be the following algorithm:

```
i = 1
while (i < n) {
  if (h_{i+1} ≥ e_{i+1} + e_i) {
    choose no assignment in week i,
    choose the hard assignment in week i+1,
    i = i + 2
  }
  else {
    choose the easy assignment in week i,
    i = i + 1
  }
}
```

Give an instance in which this algorithm does not return the optimal solution. Also say what the optimal solution is (and its value) and what the algorithm finds instead.

In general, a case where $h_{i+2} + e_i > h_{i+1} + e_{i+2}$ and $h_{i+1} > e_i + e_{i+1}$ causes the algorithm to fail to return the optimal solution. The algorithm would say to do not do any assignments in week i, and then do the hard assignment in week $i + 1$. However, this is not the optimal solution, as this stops the user from getting the points that could be achieved by h_{i+2} and e_i.

Specifically, the instance we will calculate goes as follows for a semester of $n = 3$ weeks:

Week	**Easy Pts**	**Hard Pts**
1	5	8
2	5	11
3	3	12

The maximum possible points that a person could as per the specs of this problem is 17, where they do the easy assignment in Week 1, no assignment in Week 2, and the hard assignment in Week 3. However, this algorithm would calculate that the easy assignments in Week 1 and 2 are less than the hard assignment in Week 2, so the algorithm would say to do the week 2 hard assignment, which disallows from completing the hard assignment in Week 3. Thus, they would do the easy assignment and get 14 points, and not the optimal 17.

(b) (35 points) Give an algorithm with $O(n)$ worst case running time which takes as input the values $e_1, \ldots, e_n$ and $h_1, \ldots, h_n$ and returns the value of the optimal schedule. Prove its correctness and running time.

Algorithm Description:

Let $OPT(i)$ denote the maximum number of points a student can earn in the first i weeks. The pseudocode for the algorithm is as follows:

```
OPT(0) = 0;
OPT(1) = e_1;

for i = 2 to n:
    OPT(i) = max((OPT(i-1) + e_i), (OPT(i-2) + h_i))
```

This algorithm iterates through the weeks and determines the optimal solution based on the already determined optimal solutions for previous weeks, since $OPT(i)$ cannot be determined until $OPT(0)$ through $OPT(i-1)$ have been calculated. For all i, we save $OPT(i)$ in an array for easy access, for easy access, so $OPT(0)$ pulls the optimal solution for 0 weeks located in the 0th position of the array, and so on.

Correctness Proof:

We prove correctness by strong induction.

Base case. We first consider $n = 0$. Since there are no weeks of assignments, the maximum number of points earned is 0. We now consider when $n = 1$. In this case, the algorithm has 1 option, e_1, since the hard assignment cannot be completed in Week 1 and so the optimal solution is the value of e_1. Therefore, this algorithm returns correctly.

Inductive case. We assume that this algorithm correctly computes the optimal value $OPT(j)$ for all Weeks $j < i$. We prove that the next iteration of the loop will then find the optimal solution for Week i.

For the ith week, there are two cases:

(1) $OPT(i-1) + e_i \geq OPT(i-2) + h_i$. In this case, the optimal solution is to do the easy assignment in Week i.
(2) $OPT(i-1) + e_i < OPT(i-2) + h_i$. In this case, the optimal solution is to skip Week $i-1$ and do the hard assignment in Week i.

Since all assignments are positive, the optimal solution for Week i involves completing either the easy or the hard assignment in Week i, since skipping Week i will always return a lesser solution than doing e_i, so skipping Week i is never the optimal solution. Therefore, the 2 options above represent the only possible optional solutions for Week i. This means our choices for the optimal solution are relegated to these two options.

If e_i is completed in Week i, the maximum possible score is $OPT(i-1) + e_i$, since there are no restrictions on what can be completed in the first $i-1$ weeks. By the inductive hypothesis, $OPT(i-1)$ returns the optimal solution for those weeks. Suppose, instead, we choose h_i. There is a restriction on the hard assignment, since this means no assignment can be completed in Week $i-1$. Therefore, the maximum possible solution is $OPT(i-2)+h_i$. By the inductive hypothesis, $OPT(i-2)$ returns the optimal solution for Weeks 1 to $i-2$. Since these 2 cases represent the possibilities for the optimal solution, taking their maximum would therefore give the optimal solution for the first i weeks. Therefore, the algorithm correctly finds $OPT(i)$.

In proving the base cases and the inductive case for this algorithm, we have proven that it will correctly return the optimal solution for any n.

Runtime Proof:

We analyze the algorithm above. Assigning $OPT(0)$ and $OPT(1)$ takes 2 constant-time assignments, amounting to $O(1)$. Then, the loop executes at most $n-1$ times. Within the loop, There are 2 array accesses by index to $OPT(i-1)$ and $OPT(i-2)$, 2 arithmetic computations, 1 relational operation, and 1 assignment operation. These are all constant-time operations. Therefore, each loop takes $O(1)$ time. Since the loop repeats $n-1$ times, the overall time complexity of the algorithm is $O(n-1)$, or $O(n)$.

2 Mobile Business (50 points)

Let's say that you have a great idea for a new food truck, and in order to save money you decide to run it out of your RV so you can live where you work. Each day i there is some demand for your food in Baltimore and some demand in Washington – let's say you would make B_i dollars by being in Baltimore and W_i dollars by being in Washington. However, if you wake up in one city (due to being there the previous day) and want to serve in the other city, it costs you M dollars to drive there.

The goal in this problem is to devise a maximum-profit schedule. A schedule is simply an assignment of locations to days – for each day i, the schedule says whether to serve in Baltimore

or Washington. The profit of a schedule is the total profit you make, minus M times the number of times you have to move between cities. For the starting case, you can assume that on day 1 you wake up in Baltimore.

For example, let $M = 10$ and suppose that $B_1 = 1, B_2 = 3, B_3 = 20, B_4 = 30$ and $W_1 = 50, W_2 = 20, W_3 = 2, W_4 = 4$. Then the profit of the schedule ⟨Washington, Washington, Baltimore, Baltimore⟩ would be $W_1 + W_2 + B_3 + B_4 - 2M = 100$, where one of the M's comes from driving from Baltimore to Washington on day 1, and the other comes from driving from Washington to Baltimore on day 3. The profit of the schedule ⟨Washington, Baltimore, Baltimore, Washington⟩ would be $W_1 + B_2 + B_3 + W_4 - 3M = 50 + 3 + 20 + 4 - 30 = 47$.

Given the fixed driving cost M and profits $B_1, \ldots B_n$ and $W_1, \ldots, W_n$, devise an algorithm that runs in $O(n)$ time and computes the profit of an optimal schedule. As always, prove correctness and running time.

Algorithm Description:

We maintain two optimal values – $B(i)$ and $W(i)$. Let $B(i)$ denote the maximum profit achievable on days 1 to i, if we serve in Baltimore on day i. Let $W(i)$ denote the maximum profit achievable on days 1 to i, if we serve in Washington on day i.

The pseudocode for the algorithm goes as follows:

```
W(1) = W_1 - M;
B(1) = B_1;
OPT(1) = max(W(1), B(1));

for i = 2 to n:
    B(i) = max((B(i-1) + B_{i}), (W(i-1) - M + B_{i}))
    W(i) = max((W(i-1) + W_{i}), (B(i-1) - M + W_{i}))
    OPT(i) = max(B(i), W(i))
```

Much like in the last problem, this algorithm maintains arrays W, B, and OPT, which allows for $O(1)$ access to each of the past calculations.

Correctness Proof:

We use a proof by strong induction.

Base case. We consider the case where $n = 1$. Since we wake up in Baltimore, we have 2 options per the problem specs. We can either sell in Baltimore, or pay the gas cost and sell in Washington. Thus, the possible optimal solutions are B_1 and $W_1 - M$. Therefore, the optimal solution is the maximum of these two, which is what the code determines for the optimal solution $OPT(1)$.

Inductive case. We assume that this algorithm correctly computes the optimal value $OPT(j)$ for all days $j < i$. We prove that the next iteration of the loop will then find the optimal solution for day i.

TODO

Runtime Proof:

We analyze the algorithm above. Assigning $B(1)$, $W(1)$, and $OPT(1)$ takes 3 constant-time assignments, amounting to $O(1)$. Then, the loop executes at most $n - 1$ times. Within the loop, for each i, There are 4 array accesses to $B(i-1)$ and $W(i-1)$ (twice each), 4 arithmetic computations, 3 relational operations, and 3 assignment operations. These are all constant-time

operations. Therefore, each loop takes $O(1)$ time. Since the loop repeats $n-1$ times, the overall time complexity of the algorithm is $O(n-1)$, or $O(n)$.